

【机房布局】

题目描述

小明正在规划一个大型 **数据中心** 机房，为了使得机柜上的机器都能正常满负荷工作，需要确保在每个机柜边上至少要有有一个电箱。

为了简化题目，假设这个机房是一整排，M表示机柜，I表示间隔，请你返回这整排机柜，至少需要多少个电箱。 如果无解请返回 -1 。

输入描述

```
cabinets = "MIIM"
```

其中 M 表示机柜，I 表示间隔

输出描述

```
2
```

表示至少需要2个电箱

备注

- $1 \leq \text{strlen}(\text{cabinets}) \leq 10000$
- 其中 `cabinets[i] = 'M' 或者 'I'`

用例

输入	MIIM
----	------

输出	2
说明	无

输入	MIM
输出	1
说明	无

输入	M
输出	-1
说明	无

输入	MMM
输出	-1
说明	无

输入	I
输出	0
说明	无

题目解析

本题其实只要朝一个方向优先放电箱即可，但是通常我们习惯从左向右遍历，因此这里优先将电箱放在机柜的右边。

为什么要将电箱优先放在右边呢？比如下面这个例子：



I M I M

优先将机柜放在红色I位置，则最终只需要一个电箱。如果将机柜放绿色I位置，则最终需要两个电箱。

再比如下面这个例子

M I M I

由于我们是从左往右遍历，因此第一个机柜M，优先将电箱放在它的右边。

但是，如果机柜的右侧放不了电箱呢？比如

I M M I

此时，我们需要看机柜的左侧能不能放，如果能放，则放，不然对应机柜就没有配置电箱了。

如果机柜的左右两侧都放不了电箱呢？

M M I

此时应该直接返回-1。

另外，还有一个重要的点就是：

如果机柜的右侧可以放电箱，比如第 i 个位置是机柜，第 $i + 1$ 个位置是间隔，则我们可以将电箱放到第 $i + 1$ 个位置上，此时：第 $i + 2$ 个位置是什么还需要关注吗？

比如：

M I M I

上面红色的M还需关注放不放电箱吗？答案是不需要，因为他必然有一个电箱了。

此时我们可以直接跳到i+3位置 重新开始 上面的判断。

Java算法源码

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String str = sc.next();
8
9         System.out.println(getResult(str));
10    }
11
12    public static int getResult(String str) {
13        int n = str.length();
14
15        // ans记录用了几个电箱
16        int ans = 0;
17
18        for (int i = 0; i < n; i++) {
19            // 如果当前是机柜
20            if (str.charAt(i) == 'M') {
21                // 则优先将电箱放到该机柜的右边，如果机柜右边有间隔I的话
22                if (i + 1 < n && str.charAt(i + 1) == 'I') {
23                    ans++;
24                    // 如果放成功了，即当前第 i 个位置是机柜，第i+1个位置是 电箱，那么第i+2个位置无论是机柜还是间隔都无所谓，下次我们直接从第i+3位置开始判断
25                    i += 2; // 这里 i += 2结合下轮for循环的i++，就是i += 3
26                }
27                // 如果当前第i位置的机柜右边无法放入电箱，则只能将电箱放到其左边第i-1位置，但是第i-1位置必须是间隔I
28                else if (i - 1 >= 0 && str.charAt(i - 1) == 'I') {
29                    ans++;
30                }
31                // 如果当前机柜左右都无法放入电箱，则返回-1
```

```

32         else {
33             ans = -1;
34             break;
35         }
36     }
37 }
38
39 return ans;
40 }
41 }

```

JavaScript 算法源码

```

1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5      input: process.stdin,
6      output: process.stdout,
7  });
8
9  rl.on("line", (line) => {
10      console.log(getResult(line));
11  });
12
13  function getResult(str) {
14      const n = str.length;
15
16      // ans记录用了几个电箱
17      let ans = 0;
18      for (let i = 0; i < n; i++) {
19          // 如果当前是机柜
20          if (str[i] == "M") {
21              // 则优先将电箱放到该机柜的右边, 如果机柜右边有间隔I的话

```

运行

```

22     if (i + 1 < n && str[i + 1] == "I") {23         ans++;
24         // 如果放成功了，即当前第 i 个位置是机柜，第i+1个位置是 电箱，那么第i+2个位置无论是机柜还是间隔都无所谓，下次我们直接从第i+3位置开始判断
25         i += 2; // 这里 i += 2结合下轮for循环的i++，就是i += 3
26     }
27     // 如果当前第i位置的机柜右边无法放入电箱，则只能将电箱放到其左边第i-1位置，但是第i-1位置必须是间隔I
28     else if (i - 1 >= 0 && str[i - 1] == "I") {
29         ans++;
30     }
31     // 如果当前机柜左右都无法放入电箱，则返回-1
32     else {
33         ans = -1;
34         break;
35     }
36 }
37 }
38
39 return ans;
40 }

```

Python算法源码

```

1  # 输入获取
2  s = input()
3
4
5  # 算法入口
6  def getResult(s):
7      n = len(s)
8
9      # ans记录用了几个电箱
10     ans = 0
11     i = 0
12     while i < n:

```

```

13 |         # 如果当前是机柜
14 |         if s[i] == 'M':
15 |             # 则优先将电箱放到该机柜的右边，如果机柜右边有间隔I的话
16 |             if i + 1 < n and s[i + 1] == 'I':
17 |                 ans += 1
18 |                 # 如果放成功了，即当前第 i 个位置是机柜，第i+1个位置是 电箱，那么第i+2个位置无论是机柜还是间隔都无所谓，下次我们直接从第i+3位置开始判断
19 |                 i += 2 # 这里 i += 2结合后面的i +=1，就是i += 3
20 |                 # 如果当前第i位置的机柜右边无法放入电箱，则只能将电箱放到其左边第i-1位置，但是第i-1位置必须是间隔I
21 |                 elif i - 1 >= 0 and s[i - 1] == 'I':
22 |                     ans += 1
23 |                 # 如果当前机柜左右都无法放入电箱，则返回-1
24 |                 else:
25 |                     ans = -1
26 |                     break
27 |             i += 1
28 |
29 |     return ans
30 |
31 |
32 | # 算法调用
33 | print(getResult(s))

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <string.h>
3 |
4 | #define MAX_LEN 20000
5 |
6 | int main() {
7 |     char s[MAX_LEN];
8 |     gets(s);
9 |
10 |    // ans记录用了几个电箱
11 |    int ans = 0;
12 |

```

```

13     for (int i = 0; i < strlen(s); i++) {
14         // 如果当前是机柜
15         if (s[i] == 'M') {
16             if (i + 1 < strlen(s) && s[i + 1] == 'I') {
17                 // 则优先将电箱放到该机柜的右边，如果机柜右边有间隔I的话
18                 ans++;
19                 // 如果放成功了，即当前第 i 个位置是机柜，第i+1个位置是 电箱，那么第i+2个位置无论是机柜还是间隔都无所谓，下次我们直接从第i+3位置开始判断
20                 i += 2; // 这里 i += 2结合下轮for循环的i++，就是i += 3
21             } else if (i - 1 >= 0 && s[i - 1] == 'I') {
22                 // 如果当前第i位置的机柜右边无法放入电箱，则只能将电箱放到其左边第i-1位置，但是第i-1位置必须是间隔I
23                 ans++;
24             } else {
25                 // 如果当前机柜左右都无法放入电箱，则返回-1
26                 ans = -1;
27                 break;
28             }
29         }
30     }
31
32     printf("%d", ans);
33
34     return 0;
35 }

```

C++算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      string s;
7      cin >> s;
8
9      // ans记录用了几个电箱
10     int ans = 0;

```



```
11 |
12 |     for (int i = 0; i < s.length(); i++) {
13 |         // 如果当前是机柜
14 |         if (s[i] == 'M') {
15 |             if (i + 1 < s.length() && s[i + 1] == 'I') {
16 |                 // 则优先将电箱放到该机柜的右边，如果机柜右边有间隔I的话
17 |                 ans++;
18 |                 // 如果放成功了，即当前第 i 个位置是机柜，第i+1个位置是 电箱，那么第i+2个位置无论是机柜还是间隔都无所谓，下次我们直接从第i+3位置开始判断
19 |                 i += 2; // 这里 i += 2结合下轮for循环的i++，就是i += 3
20 |             } else if (i - 1 >= 0 && s[i - 1] == 'I') {
21 |                 // 如果当前第i位置的机柜右边无法放入电箱，则只能将电箱放到其左边第i-1位置，但是第i-1位置必须是间隔I
22 |                 ans++;
23 |             } else {
24 |                 // 如果当前机柜左右都无法放入电箱，则返回-1
25 |                 ans = -1;
26 |                 break;
27 |             }
28 |         }
29 |     }
30 |
31 |     cout << ans;
32 |
33 |     return 0;
34 | }
```